

AUGUR: Specification Oracles for Network Protocol Fuzzers

Abstract—Stateful network protocol fuzzers are widely used for uncovering software bugs in network protocol implementations: protocol specific fuzzers rely on test oracles to identify logical bugs, whereas grey-box fuzzers leverage lightweight instrumentation to uncover crashes. Both generate sequence of messages as test inputs to drive the stateful execution of protocol implementations under test. While protocol-specific fuzzers equipped with test oracles can identify logical bugs, general-purpose network-protocol fuzzers cannot. Additionally, the frequent unavailability of faithful test oracles leads to complicated manual triaging: Most fuzzers flag inputs without considering whether they indeed demonstrate an expected property violation (*i.e.*, a true logical bug), increasing the number of erroneously-flagged inputs to triage. In both scenarios, users are either (1) forced to develop custom test oracles to identify logical bugs and manually triage fuzzer-flagged inputs, or (2) unable to identify logical bugs at all. We address these issues by presenting AUGUR, a highly configurable property-aware approach that automatically detects logical bugs during fuzzing runtimes. We evaluate AUGUR via integration with 4 stateful network protocol fuzzers across 17 implementations of 6 protocols (WPA3, DNS, OpenSSH, DTLS, RTSP, FTP), out of which two fuzzers are protocol-specific black-box fuzzers (SAECRED, ResolverFuzz) and two are general-purpose network protocol grey-box fuzzers (AFLNET, SNPSFuzzer). AUGUR uncovered 14 semantic bugs across all 4 fuzzers, out of which 6 were previously unknown.

I. INTRODUCTION

Network protocols are critical infrastructure for modern cyberspace, and flaws in their implementations can have far-reaching consequences [14], [46]. A popular approach to avoiding such undesirable outcomes is to use *automated software testing techniques* to identify bugs in protocol implementations before they are deployed. Although fuzz testing (a.k.a. fuzzing) is a powerful automated software testing technique that has been prolific in uncovering critical bugs on a wide range of software systems [51], [16], it cannot always be directly applied to testing protocol implementations. This is because protocols often exhibit stateful behavior, have highly-structured input formats, use cryptography, or have complex semantics. Such unique challenges inspired the development of a wide range of fuzzers specifically designed to target protocol implementations [51], [16], [38], [34], [39], [42], [17], [19], [18], [37], [8], [53], [24], [36]. *This paper focuses on this specialized class of fuzzers* (referred to as protocol fuzzers).

The current impressive spread of protocol fuzzers can be broadly classified along several dimensions, including their visibility into the target (black-box vs. gray-box) [9], [18], [19], [4], their protocol awareness (protocol-agnostic vs. -specific) [11], [53], [8], [39], [52], [51], [38], their test-case generation strategy (valid, malformed, or hybrid) [38], [19],

[44], [39], and the classes of vulnerabilities they are designed to detect (memory vs. logic) [52], [9], [39], [23]. In the context of this paper, we are interested in one of the dimensions: *protocol awareness*.

Protocol-agnostic fuzzers, dubbed FUZZERS_{PA}'s [38], [24], [34], [9], [15], [3], [28], [37] impressive generality comes at the price of only being able to discover memory-safety bugs that are typically manifested as crashes of the System Under Test (SUT). However, they are incapable of detecting logic errors that do not manifest as crashes or induce memory-safety violations that are detectable through address sanitizers [51], [16], [38], [34], [24], [48], [3]. This is because detecting such logic bugs requires understanding of the protocol's semantics; unavailable to FUZZERS_{PA}. While protocol-specific fuzzers, FUZZERS_{PS} [39], [52], [4], [20], [30], [18], [11], [53], can discover logic bugs due to their knowledge of the given protocol semantics, they lack generalizability. *This paper presents a principled framework and accompanying tooling dubbed AUGUR that enables FUZZERS_{PA} to discover semantic bugs and allows FUZZERS_{PS} to automatically triage flagged inputs.*

Before describing AUGUR, we precisely characterize what we mean by *semantic/logic bugs* and how existing fuzzers uncover such bugs. We focus on only those semantic bugs whose violations are *observable* through the SUT's input-output interface. Precisely, these classes of properties are known as *extensional* properties [31]. Our scope of semantic bugs includes both *stateless properties* (*e.g.*, the server should not respond with a malformed packet) and *stateful properties* (*e.g.*, the server should not respond with a malformed packet after receiving a client hello). We exclude from our scope semantic bugs that require reasoning about internal protocol states that do not have observable manifestations.

To discover logic bugs in protocol implementations, one generally requires a *test oracle* that can adjudicate whether the output (a sequence of protocol messages) generated by the SUT for a *given test input* (also, a sequence of protocol messages) is an *expected* behavior. A test oracle is typically realized in one of the following three ways: (1) *cross-checking oracles*; (2) *reference implementation oracles*; (3) *specification oracles*. Cross-checking oracles appear in the context of *differential testing* where one test input is fed into diverse implementations of the same protocol, and the outputs of the different implementations are compared, where discrepancies are flagged as potential bugs [21], [10]. If the same bug appears on all the SUTs, that bug will evade detection by cross-checking oracles [39], [7]. In contrast, a reference implementation oracle is a full implementation that captures

the expected behavior of the protocol, and it is used as a reference to compare the SUT’s behavior against. Such oracles can be formally verified, but the effort required to develop such high-fidelity oracles is often prohibitively high [13], [49]. Such oracles often suffer from false positives as the SUT’s behavior may diverge from the oracle’s expectations without violating any protocol properties. Specification oracles focus on the desirable properties that the protocol aims to achieve, avoiding the problem of false positives.

AUGUR is a lightweight approach of constructing a specification oracle for a protocol from its *extensional* specification. It is designed to optimize the following three dimensions: *expressivity* in terms of the classes of identifiable bugs, *efficiency* in terms of computational overhead, and *portability* between fuzzers and SUTs. AUGUR expects protocol properties to be captured as quantifier-free, past-only, first-order linear temporal logic (QF-FO-PLTL) formulas [26]. This logic is expressive enough to capture most protocol properties, both stateless and stateful.

AUGUR exposes an API which can be called by any FUZZERS_{PA} to incorporate AUGUR’s capability of discovering semantic bugs. AUGUR is lightweight, running alongside (but separately from) the fuzzer and SUT processes. Precisely, AUGUR plays the role of an external *observer* and uses techniques from runtime verification [6], [5], [50] to monitor the interactions between the fuzzer and the SUT at runtime and discover violations of the specified properties (*i.e.*, semantic bugs). In addition, the choice of QF-FO-PLTL as the specification language enables AUGUR to employ an incremental formula evaluation algorithm that requires only *linear memory and time* in the size of the formula. This allows AUGUR to support fuzzer efficiency features like *memory snapshots* with minimal overhead. AUGUR requires no modification to the SUT and the internal logic of the fuzzer (only a simple I/O hook). This is in contrast to previous LTL-based work [32] that requires instrumentation of the SUT. The fundamental tradeoff is that prior work [32] allows one to capture *intensional* properties that reference internal system state, while AUGUR only supports *extensional* properties that reference aspects of the observable system I/O [31]. We argue that restricting to extensional properties greatly enhances portability while still allowing one to express most relevant properties. AUGUR is agnostic to the particular strategy that the underlying fuzzer uses to manage state (*e.g.*, with a state oracle or state inference). AUGUR can also be used by FUZZERS_{PS} to automatically triage flagged inputs by excluding false positives that do not violate any protocol property.

The requirements for integrating AUGUR with a fuzzer are as follows: ❶ *Monitor integration*: A fuzzer must use AUGUR’s API to give AUGUR visibility of the test inputs and received outputs from the SUT at runtime. ❷ *Property specification extraction*: AUGUR exposes a DSL for specifying the extensional properties of the protocol being monitored. Requirement ❶ must only be implemented one time per fuzzer, while requirement ❷ must only be implemented one time per protocol. After these one-time efforts, AUGUR can be used for

Aspect	Specification Oracles	Reference Oracles
<i>Upfront Effort</i>		
Specification extraction	Comparable	Comparable
Packet parsing	Comparable	Comparable
<i>Engineering Burden (Cost)</i>		
Explicit state modeling	None	High
Transition-level encoding	Optional	Required
Level of complexity	Low	High
<i>Effectiveness (Benefit)</i>		
Detection of semantic violations	High	Moderate
Robustness to spec ambiguity	High	Low
False positive rate	Low	High
<i>Practical Deployment</i>		
Cross-implementation applicability	High	High
Ease of integration	High	High

TABLE I: Comparison between specification and reference implementation oracles. Green denotes positive traits; red denotes negative traits; orange denotes neutral traits.

semantic bug extraction across multiple protocol implementations with *no additional changes* to either AUGUR, the fuzzer, or the SUT. We argue that integrating AUGUR with a fuzzer requires less effort than developing a high-fidelity reference implementation oracle (see Table I). The most challenging step of AUGUR is gathering the protocol properties being monitored, which one has to do anyway for developing a reference implementation oracle.

Results. For empirical evaluation, we integrate AUGUR with two FUZZERS_{PS} (Saecred, ResolverFuzz) and two FUZZERS_{PA} (AFLNET, SNPSFuzzer) targeting 6 protocols (WPA3, DNS, OpenSSH, DTLS, RTSP, FTP) across 17 implementations. Our evaluation demonstrates that AUGUR effectively identifies semantic bugs across all fuzzers, enabling FUZZERS_{PA} to uncover 6 semantic bugs that were previously out of reach and identify 8 previously-known semantic bugs (reported by FUZZERS_{PS}), showcasing AUGUR’s versatility and effectiveness in extending the scope of prior fuzzing techniques.

Contributions. In summary, we contribute the following:

- (i) We introduce AUGUR, a protocol property-aware approach to enable FUZZERS_{PA} to identify semantic bugs and allow FUZZERS_{PS} to automatically triage flagged inputs.
- (ii) We provide an implementation of AUGUR that is lightweight, configurable, and operates in either online or offline modes.
- (iii) We describe our experimental evaluation of AUGUR across 6 protocols and 17 implementations: AUGUR uncovers 14 semantic bugs (out of which 6 were previously unknown) while greatly reducing manual triaging effort.

All identified semantic bugs were responsibly disclosed, resulting in patches across multiple affected implementations.

II. MOTIVATION FOR AUGUR

This section presents AUGUR's preliminaries and motivation.

Abstract Model of Stateful Protocols. Abstractly, a stateful protocol can be represented as a state machine $\mathcal{M}$, which is tuple of the form $\langle S, \mathcal{I}, \mathcal{O}, T, S_0 \rangle$, where S is the set of possible states, $\mathcal{I}$ is the set of all input messages (both well-formed and malformed), $\mathcal{O}$ is the set of output messages, $T \subseteq S \times \mathcal{I} \times S \times \vec{\mathcal{O}}$ is the (left-total) transition relation, and $S_0 \subseteq S$ is the set of initial states. $\vec{\mathcal{O}}$ is the set of finite sequences of messages from $\mathcal{O}$ that may be generated by the protocol in response to an input message from $\mathcal{I}$. Although the input and output messages (*i.e.*, $\mathcal{I}$ and $\mathcal{O}$) are intentionally left abstract in this presentation, in practice, they can be viewed as possibly nested records where some fields can be variable length. An *execution* of the protocol $\mathcal{M}$, represented as σ , is a sequence of tuples of the following form: $(\langle s_0, m_0, [o_1, o_2, \dots, o_m] \rangle, \langle s_1, m_1, [o_1, o_2, \dots, o_t] \rangle, \dots, \langle s_n, m_n, [o_1, o_2, \dots, o_u] \rangle)$. We use σ_i to represent the i -th element of an execution σ . An execution σ of the form above is *valid* if it satisfies the following conditions: (1) $s_0 \in S_0$; (2) for all $i \in [0, \dots, n-1]$, if $\sigma_i = \langle s_i, m_i, [a_1, a_2, \dots, a_x] \rangle$ and $\sigma_{(i+1)} = \langle s_{(i+1)}, m_{(i+1)}, [b_1, b_2, \dots, b_y] \rangle$, then $(s_i, m_i, s_{(i+1)}, [a_1, a_2, \dots, a_x]) \in T$.

This protocol model is general enough to capture many real-world protocols and their implementations including SSL/TLS, 4G/5G, BLE, and Wi-Fi SAE. Our work and the following discussion is equally applicable to both network and application protocols, including ones that use cryptography.

The Idealized Problem for FUZZERS_{PS} and FUZZERS_{PA}. Given a protocol design P with abstract protocol model $\mathcal{M}_P = \langle S_d, \mathcal{I}, \mathcal{O}, T_d, S_0^d \rangle$, a sequence of desired properties $\Psi = \{\psi_0, \psi_1, \dots, \psi_n\}$, and an implementation under test I of P with abstract protocol model $\mathcal{M}_I = \langle S_I, \mathcal{I}, \mathcal{O}, T_I, S_0^I \rangle$, both FUZZERS_{PS} and FUZZERS_{PA} aim to generate a sequence $s = [b_0, b_1, b_2, \dots]$ of inputs with the following properties: (1) each b in s has the form $[m_0, m_1, \dots, m_n]$ where $m_j \in \mathcal{I}$ for all j ; (2) for every input b from s to $\mathcal{M}_P$ and $\mathcal{M}_I$ and receive the valid traces σ_P and σ_I , respectively, σ_I satisfies all the properties in Ψ while σ_P falsifies at least one of them.

A. Motivation of AUGUR for FUZZERS_{PS}

Most FUZZERS_{PS} have the following main logical components for discovering logical bugs: (1) a *state-aware input generator*; (2) a *test executor*; (3) a *deviation checker*. The state-aware input generator generates an input $M = [m_0, m_1, m_2, \dots, m_n]$ where each $m_i \in \mathcal{I}$. Although $m_i \in \mathcal{I}$, it is not necessarily well-formed. The input generator generates unexpected inputs either by mutating a well-formed input or by generating a new input from malformed packet structure (*i.e.*, often known as grammar-based fuzzers [22], [39]). The test executor feeds the input M to the protocol implementation, and obtains the output $O = [\vec{o}_0, \vec{o}_1, \vec{o}_2, \dots, \vec{o}_n]$ where each element of $\vec{o}_i$ is in $\mathcal{O}$. The deviation checker then checks to see if the output O deviates from the expected output

$O^* = [\vec{o}_0^*, \vec{o}_1^*, \vec{o}_2^*, \dots, \vec{o}_n^*]$ in at least one position. To know what the expected output O^* is for each given input FUZZERS_{PS} need to rely on a *test oracle* that provides O^* . The following are the predominant approaches for constructing test oracles for protocol implementations. **① Cross-checking oracles:** One can cross-check the output of different implementations (or, different versions of the same implementation) of the same protocol. The insight is that if the outputs of two implementations deviate for the same input, then at least one of them is incorrect. Unfortunately, this is not effective when all the cross-checked implementations are incorrect, as shown in Dar *et al.* [39] for the Wi-Fi SAE protocol. **② Reference implementation oracles:** One popular approach is to develop a full-fledged implementation of the protocol-under-test, which is then used as a reference oracle during fuzzing. However, the fidelity of such reference implementation oracles can substantially influence its effectiveness. Formal verification of the implementation can be used to achieve high assurance, but this generally requires substantial human effort and expertise. One subtle point about full implementations as reference implementation oracles is that they have to commit to specific implementation choices, which may not be the same as those of the implementation under test causing false positives. **③ Specification oracles:** AUGUR follows a specification oracle paradigm, in which rather than a full-fledged implementation one essentially encodes the desired guarantees of the protocol under test as checkable assertions. This approach has the following advantages: (1) it has less false positives given the checkable assertions precisely capture the desired guarantees; (2) it requires comparatively less human effort than a reference implementation oracle as one does not need to develop a full-fledged implementation of the protocol.

Erroneously Flagging Inputs as Bugs. The above discussion shows that the ability of FUZZERS_{PS} to correctly *flag an input as a bug* rests on the correctness of their reference implementation oracle. However, if equipped with a correct reference implementation oracle, FUZZERS_{PS} do not take into account the expected or desirable functional and security properties of the protocol under test (*i.e.*, Ψ). Rather, they rely on deviation of the observed output message sequences as a proxy for the correctness of the protocol implementation. However, a local transition-level deviation does not necessarily entail a violation of the properties in Ψ as shown, for example, by Dar *et al.* [39] for the Wi-Fi SAE protocol. Specifically, the SAE Handshake consists of a commit and confirm phase, where both parties commit before confirming. This implies that once a client sends a commit frame, it waits for the AP to respond with its own commit frame, after which the client generates session keys and sends a confirm frame. In practice, after APs commit, they immediately begin transmitting confirm frames, allowing clients to check that the AP correctly generated the session key. While this behavior may seem unexpected, it leads to no property violations. Such behaviors are implementation-specific and are difficult to predict with heuristic approaches. **Automated Triaging of Erroneously Flagged Inputs.** To address the above issue, FUZZERS_{PS} often require a human

expert to manually triage the flagged inputs to determine whether they indeed indicate the presence of bugs and, if so, which of them should be analyzed and fixed first. Triageing is a time-consuming and error-prone process, especially when the number of flagged inputs is substantial and many of them are erroneously flagged. In our context, the process of triaging has following two goals: (a) given a flagged input, automatically determine whether it violates any of the expected properties or demonstrates a security vulnerability; (b) out of all the correctly flagged inputs, which high-priority bugs should be investigated first to identify their root cause. Unfortunately, there are no existing mechanisms to automate the triaging process; motivating the design of AUGUR.

B. Motivation of AUGUR for FUZZERS_{PA}

How FUZZERS_{PA} detect bugs. FUZZERS_{PA} typically have foundations in the standard coverage-guided, mutation-based greybox world. These techniques instrument binaries to communicate coverage information on the SUT. At each iteration of the fuzzing loop, a set of inputs is randomly sampled, and those that increase coverage are prioritized and mutated to produce new inputs. This paradigm has proven highly effective for stateless or weakly stateful programs, and forms the basis of widely used fuzzers such as AFL and its derivatives. Additionally, network protocols’ inherently statefulness means that naive coverage guidance is often insufficient to explore deep protocol logic. To address this challenge, prior work has proposed several techniques for incorporating state awareness into greybox fuzzing. A common approach is *implicit state inference*, where protocol states are approximated using observable execution features such as response codes, message types, or coverage profiles; AFLNET [38], for example, clusters server responses and coverage signals to infer a coarse-grained protocol state machine that guides input scheduling and mutation [38]. Despite this, existing greybox approaches typically cannot detect semantic bugs that do not manifest as crashes or memory safety errors.

Lifting FUZZERS_{PA} to semantic bugs. FUZZERS_{PA} have a minimal barrier to entry, leading to their wide adoption (*e.g.*, popular use in CI/CD workflows [45]). While FUZZERS_{PS} may be more effective, decision-making often prefers efficiency over comprehensiveness. Ideally, FUZZERS_{PA} could also identify semantic bugs without any extra cost (*i.e.*, challenging protocol-specific logic). AUGUR does not fully achieve this ideal: it requires a property specification (as temporal formulas) and a protocol-specific predicate evaluator that maps concrete packets to predicate values. However, we argue that this is a small cost compared to state-of-the-art protocol-specific approaches. To the best of our knowledge, AUGUR is the first LTL-(variant-)based fuzzing approach, agnostic to both the implementation under test and the fuzzer’s internal mechanisms, while still enabling semantic bug detection and triage.

III. REFERENCE IMPLEMENTATION ORACLE VS. SPECIFICATION ORACLE

Recall that, AUGUR uses a *specification oracle*: an oracle that evaluates the behavior of the system under test with respect to a declarative logical specification. This section illustrates the key differences between specification and reference implementation oracles; see Table I for a summary. While both approaches require extracting semantics from specifications, we argue that reference implementation oracles incur significantly higher engineering cost while providing weaker and less reliable signals for identifying semantic bugs.

A. Implementation Effort

Both approaches require effort to extract protocol semantics from specifications. However, the form in which these semantics must be encoded differs significantly.

Reference implementation oracle requirements. Reference implementation oracles must encode protocol behavior as an explicit state machine. This requires modeling valid states, transitions, and edge-case behaviors, often at a fine granularity. As a result, protocol properties are implicitly embedded within transition logic. This tight coupling between semantics and state transitions makes reference implementation oracle development both labor-intensive and error-prone, particularly in the presence of underspecified or ambiguous protocol behavior.

Specification oracle requirements. In contrast, specification oracles encode protocol semantics directly as properties over execution traces. These properties capture behavioral requirements, including temporal relationships (*e.g.* ordering constraints between messages), without requiring explicit state machine construction. By avoiding transition-level modeling, specification oracles provide succinct and direct representations of protocol behavior, resulting in lower specification and implementation overhead.

One can view the difference between a reference implementation oracle and specification oracle similarly to the difference between an *implementation* and a *specification*. In general, we argue that declaratively specifying *what* a protocol should do is less error-prone than specifying the details of *how* it should be done. After all, to generate a faithful implementation, one still has to understand the specification, so we argue that *just* generating the specification should be considerably less work in most cases.

B. Example

To illustrate benefits of specification oracles, consider implementing a reference implementation oracle (as a state machine). A fragment of such an oracle, along with a corresponding fragment for some implementation under test, is presented in Figure 1. These state machines both capture part of RFC 959 [41], which states that before the server transmits file data through a RETR command, a connection must have been established through a PORT command (considering only active-mode transfers for brevity). The reference implementation oracle state machine does exactly that; it requires a PORT command (acknowledged with a 200 by the server) before

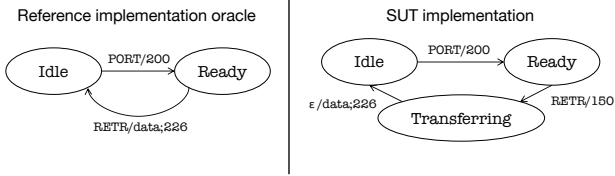


Fig. 1: FTP reference implementation oracle vs. SUT implementation example. Each transition is labeled with the form “input/output” (*i.e.*, the given transition is triggered in the corresponding state with the corresponding input and produces the corresponding output). ϵ denotes the absence of input.

moving to the **Ready** state which can then process a **RETR** command. Upon processing a **RETR**, the server responds with the requested data and a 226. However, the implementation responds to the **RETR** with a 150 before responding with the data and 226. Here, the 150 indicates an *optional* preliminary positive reply before responding with the data. Hence, an approach comparing the output of the reference implementation oracle to the implementation would notice a discrepancy and raise a spurious violation, as both are valid with respect to the RFC.

In contrast to the reference implementation oracle, the desired behavior can be concisely and precisely captured with the temporal logic property in a specification oracle: $H \text{ (data_sent} \Rightarrow Y \text{ (} O \text{ port_ok))}$ (*i.e.*, every time data is sent, there was some preceding successful **PORT** command). Here, *every* falsifying trace is guaranteed to violate the RFC.

IV. DESIGN OF AUGUR

This section presents **AUGUR**’s concrete design and design space.

A. **AUGUR**’s Design Space

We now outline the design space for the general problem of fusing runtime verification with FUZZERS_{PA} and FUZZERS_{PS} . **Property representation.** A central design decision concerns how to represent the expected system properties (*i.e.*, the protocol specification). In general, one may represent properties either using *automata* or as *formulas* in a formal logic. Among temporal logics, linear temporal logic (LTL) has seen particularly widespread adoption in the verification community [40]. Other well-known temporal logics include CTL and CTL*, which adopt a *branching-time* semantics rather than a linear-time one. While these logics are expressive, their semantics makes them less natural for trace-based reasoning and runtime verification. They are therefore less commonly used in such settings. LTL formulas admit equivalent representations as *Büchi automata*, which are employed by prior work [32]. However, translations from LTL to Büchi automata can incur an exponential blowup in the number of states in the worst case [47]. One could restrict attention to simpler automata models such as deterministic finite automata (DFAs), shifting from ω -regular languages to regular languages over finite traces. However, compared to past-only LTL (which also has

a well-defined semantics over finite traces), DFAs incur a (potentially) exponential blowup in memory [12]. Within the LTL family, one may further choose between *future-only*, *past-and-future*, or *past-only* temporal operators, as well as between *quantified* and *quantifier-free* fragments, and *propositional* versus *first-order* settings. Our choice of a *past-only*, *quantifier-free* fragment is motivated by the existence of time- and memory-efficient runtime verification algorithms. We also work with a first-order variant, but recover the evaluation performance of the propositional setting by relying on predicate evaluators that map predicates over symbolic terms, together with a concrete packet, to Boolean values. Once predicates are evaluated, runtime verification proceeds identically to the propositional case.

AP Representation. When working with propositional LTL, one can encode relevant properties of packets directly as atomic propositions (*e.g.*, `packet_is_well_formed`, `length_greater_than_3`). This approach yields simple and efficient runtime verification, but requires enumerating all relevant Boolean properties of interest in advance, which becomes cumbersome when properties depend on concrete packets or relationships between fields. Prior work [32] uses *events* as the atomic propositions in their LTL formulas. Intuitively, each event is a term $e(t_1, \dots, t_n)$ that evaluates to **True** if and only if event e (parameterized by terms t_1 through t_n) occurs. While event-based atomic propositions are intuitive to reason about and map naturally to protocol actions, they do not naturally support arithmetic constraints or, more generally, predicates that are not fundamentally event-based (*e.g.*, numeric ranges, comparisons between packet fields).

Trace evaluation. Prior work [32] represents properties using Büchi automata and evaluates a formula by executing the corresponding automaton over a given trace. To support liveness properties, that approach additionally checks for cycles in program states, which requires instrumentation of the SUT to expose and compare internal execution states. This instrumentation introduces nontrivial overhead and complicates integration with off-the-shelf fuzzers. Alternatively, one could employ a general-purpose model checker to determine whether a trace satisfies a given property. While effective for full-system verification, model checking introduces unnecessary computational overhead in trace-based settings and becomes increasingly expensive for richer logics. In contrast, **AUGUR** adopts a dynamic-programming-based runtime verification approach for past-only LTL that evaluates properties incrementally as the trace is produced. This approach requires only a constant amount of state per subformula and runs in $O(|\varphi|)$ time per event, where $|\varphi|$ is the size of the property. We deliberately restrict attention to safety properties, thereby avoiding both cycle detection and instrumentation of the SUT. **Liveness and safety.** **AUGUR** does not support liveness properties, which is a limitation compared to prior work [32]. However, that approach checks liveness properties by reasoning over Büchi automaton executions and detecting cycles in program states, which incurs nontrivial computational overhead. In contrast, **AUGUR** deliberately restricts attention to

safety properties in order to enable lightweight, trace-based runtime verification without cycle detection or instrumentation of the SUT. Although AUGUR does not support unbounded liveness properties, it is expressive enough to capture *bounded liveness* properties, which we argue are sufficient in practice for fuzzing-based analysis. For example, the property “every client request must eventually be followed by a server response” can be rephrased as “every client request must be followed by a server response within X timesteps,” for some fixed bound X . Such bounded liveness properties naturally align with finite executions and protocol timeouts, and they can be evaluated efficiently using our linear-time runtime verification algorithm.

Intensional vs. extensional properties. *Intensional* properties are properties that refer to internal, implementation-level state, whereas *extensional* properties are defined purely in terms of externally observable behavior. *AUGUR supports only extensional properties.* While this is a restriction, it allows AUGUR to remain independent of protocol implementation details and thereby maximizes portability across different implementations. In contrast, specifying intensional properties requires detailed knowledge of the internal structure and state of a particular implementation, which is often unavailable or impractical in real-world fuzzing scenarios. Moreover, when intensional aspects of a system are of interest, they can sometimes be inferred indirectly from extensional observations. We argue that extensional properties are typically sufficient to comprehensively test protocol implementations, a position also advocated by McMillan and Zuck [31].

B. AUGUR Architecture

We now discuss AUGUR’s high-level architecture (see Figure 2).

Input. AUGUR takes as input a *property specification file* ①, which contains the set of expected protocol properties to be considered for evaluation. Expected properties are expressed as temporal safety properties [2] captured in quantifier-free, past-only first-order linear temporal logic (QF-FO-PLTL) [26]. We choose this logic as formulas in this logic are sufficiently expressive to capture a wide range of expected properties and efficiently monitorable (both in terms of space and time) [6]. Its semantics is also well-defined on finite traces. Each protocol (but not each implementation) needs its own property specification file.

Parsing. While fuzzing, the fuzzer’s driver ② sends each incoming and outgoing packet to the parsing component, comprised of the PKTPARSER ③ and FIELDEXTRACTOR ④. Upon receiving each raw packet, the PKTPARSER parses it into an abstract syntax tree (AST). Then, the FIELDEXTRACTOR walks this AST and sets variables representing packet fields to their concrete values. Notice that before fuzzing starts, the FIELDEXTRACTOR can statically determine the set of all relevant packet fields based on the list of properties in the property specification file ①. Each protocol (but not each

implementation) needs its own PKTPARSER and FIELDEXTRACTOR.

Runtime verification. At each step, the PREEVALUATOR ⑤ incrementally receives a mapping from variables representing packet fields to concrete values from the FIELDEXTRACTOR ④. The PREEVALUATOR also receives an optional value wrapping the AST from parsing ast_o , where absence of the AST denotes that the packet did not parse. From these inputs, the PREEVALUATOR computes the truth values of all the predicates listed in the property specification file ①. For example, the PREEVALUATOR could compute the truth value of $packet_len >_{bv} 0x10$ when given a mapping that defines a concrete value for $packet_len$. More complex predicates, like $IsPacketWellFormed(ast_o)$, can be computed directly in terms of the AST (in this case, one can implement $IsPacketWellFormed$ by simply checking if ast_o is present). Once all the predicates are assigned truth values, the MONITOR ⑥ evaluates the satisfaction status of each expected property at each timestep (bottom-up). Each protocol (but not each implementation) needs its own PREEVALUATOR. The MONITOR is a one-time implementation effort, as it is easily parametrized with respect to the specific monitored properties.

Generic API. Both FUZZERS_{PS} and FUZZERS_{PA} require code changes to integrate with AUGUR, at four key locations, each corresponding to a call into AUGUR’s API (Listing 1):

```
1 void Init(spec_path) // initialize system from spec file
2 void ResetState() // reset monitor state
3 void IngestPacket(pkt) // send pkt from driver to parser
4 int SaveState() // save memory snapshot, returning
  snapshot ID
5 void RestoreState(id) // restore memory snapshot
```

Listing 1: AUGUR’s generic fuzzer API.

Init(spec_path) is called at fuzzer initialization. It loads the properties from the property specification file at $spec_path$ and prepares the FIELDEXTRACTOR and MONITOR for the target protocol.

ResetState() is called at the start of each new fuzzing execution. It reinitializes the MONITOR to a clean state, ensuring no leakage across executions.

IngestPacket(pkt) is called after each protocol packet is sent or received. It appends pkt to the parser’s queue, which parses the packet and evaluates predicates before sending the resulting mapping to the MONITOR for evaluation.

Restore(snapshot) is called at snapshot-based resets (for fuzzers that support memory snapshots, e.g., [24]). It restores the MONITOR to the state captured at $snapshot$, keeping runtime verification consistent with the fuzzer’s restored execution state.

C. Running example

We present a concrete running example based on the WPA3 SAE handshake to illustrate the end-to-end pipeline of AUGUR in conjunction with an existing fuzzer.

Property Specification File. We designed a domain-specific language (DSL) for specifying the expected properties for each

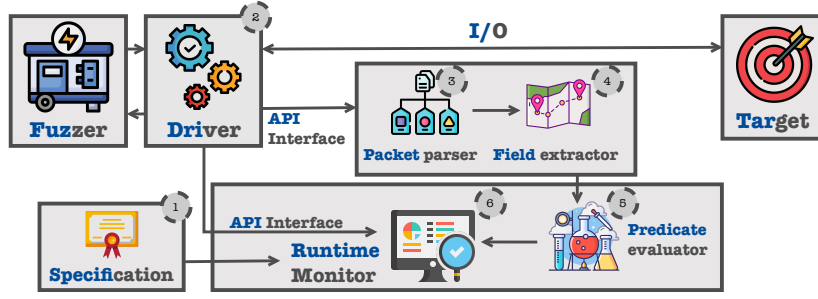


Fig. 2: The AUGUR architecture.

protocol. Our DSL is protocol-agnostic and can be used to specify properties expressed in QF-FO-PLTL [26].

```

1  enum request {
2      CLIENT_COMMIT_SUCCESS, CLIENT_COMMIT_FAIL, ...
3  };
4  enum response {
5      AP_COMMIT_SUCCESS, TIMEOUT, AP_COMMIT_ERROR, ...
6  };
7  enum support {
8      SUPPORTED, UNSUPPORTED
9  };
10 bool rg_container; bool pi_container;
11 int client_status_code; int ap_status_code; int group;
12 H (
13     Y (request = CLIENT_COMMIT_SUCCESS & rg_container &
14         not pi_container & support = SUPPORTED) ->
15         response = TIMEOUT
16 );

```

Listing 2: An example property in our DSL for WPA3 SAE.

Initialization. AUGUR initializes along with the fuzzer by loading the specification file and constructing internal data structures required to evaluate formulas. Suppose that Listing 2 is the specification provided to AUGUR. First, the MONITOR rewrites the property τ into a set of formulas that gives a name to each subformula:

$$\begin{aligned}
 a &\Leftrightarrow (\text{request} = \text{CLIENT_COMMIT_SUCCESS}) \\
 b &\Leftrightarrow (\text{rg_container} = \text{True}) \\
 c &\Leftrightarrow (\text{pi_container} = \text{False}) \\
 d &\Leftrightarrow (\text{support} = \text{SUPPORTED}) \\
 e &\Leftrightarrow (\text{response} = \text{TIMEOUT}) \\
 f &\Leftrightarrow Y (a \wedge b \wedge c \wedge d) \\
 g &\Leftrightarrow f \Rightarrow e \\
 \tau &\Leftrightarrow H g
 \end{aligned}$$

The MONITOR maintains the truth values of all subformulas using a bit vector $\langle [a], [b], [c], [d], [e], [f], [g], [\tau] \rangle$. Here, $\langle \dots \rangle$ denotes a bit vector literal, and $[x]$ denotes x 's truth value.

Input. When the WiFi fuzzer generates a test input, AUGUR processes the packet and parses it into an AST. This AST is then passed to the FIELD EXTRACTOR, which traverses the structure and assigns concrete values to the protocol variables defined in Listing 2.

Parsing. Assume a well-formed COMMIT frame. The FIELD-EXTRACTOR assigns `request` to `CLIENT_COMMIT_SUCCESS`,

extracts the rejected groups container (sets `rg_container` to `True`), observes the password identifier container's absence (sets `pi_container` to `False`), and determines that the advertised group is supported (sets `support` to `SUPPORTED`). Since no response has yet been observed, `response` is set to `RESPONSE_NOT_SET`. The resulting mapping is serialized and forwarded (along with the original AST) to the PREDEVALUATOR.

Runtime verification. The variable assignment is processed by the PREDEVALUATOR, which computes the truth value of each atomic predicate in the specification, producing a mapping of predicates to truth values that is passed to the MONITOR. For example, the predicate `response = TIMEOUT` evaluates to `False` as the `response` variable is mapped to `RESPONSE_NOT_SET`, which is not `TIMEOUT`.

The MONITOR then computes the truth value of every subformula, encoded as a bit vector. After the first step, the resulting state is $\langle 1, 1, 1, 1, 0, 0, 1, 1 \rangle$. Above, the first bit of the bit vector refers to the truth value of a , the second to the truth value of b , and so on (where 1 denotes `True` and 0 denotes `False`). Here, $[f] = 0$ as $Y e$ evaluates to 0 on the initial timestep for any expression e . Also, $[g] = 0$ as the implication $f \Rightarrow e$ is vacuously satisfied. Evaluating bottom-up, we have $[\tau] = 1$, so no violation is reported.

Output hook. Once the SUT produces an observable output, AUGUR captures the response and processes it through the same pipeline. Under correct behavior, the SUT should not respond, resulting in `response` value of `TIMEOUT`. However, several implementations incorrectly respond with a valid COMMIT frame, which sets `response` to `AP_COMMIT_SUCCESS`. This updated assignment is again evaluated through the PREDEVALUATOR and passed to the MONITOR. In the current timestep, request-related predicates take a default value of `False` because we are processing a response, not a request. In this step, f has truth value 1 because $a \wedge b \wedge c \wedge d$ held in the previous step. The updated state becomes $\langle 0, 0, 0, 0, 1, 0, 1, 0 \rangle$. Property τ is violated, and the MONITOR records the violating trace.

State management. At the end of each test case, the driver invokes the `ResetState()` API, clearing the MONITOR state so that each trace is evaluated independently. This resets the bit vector for all subformulas. For snapshot-based fuzzers such as SNPSFuzzer, AUGUR additionally supports `SaveState()`

and `RestoreState(id)` APIs, ensuring consistency between the `MONITOR` state and the execution state of the fuzzer. In other words, whenever the fuzzer saves a memory snapshot, it should also save the bit vector representing `MONITOR` state, and analogously for restoring a memory snapshot.

Iteration. This process repeats for each generated test case.

Key Observation

AUGUR operates purely as an *observer*. AUGUR does not influence input generation or mutation strategies, nor does it provide feedback to guide exploration. This enables AUGUR to be deployed as a drop-in semantic analysis layer without requiring modifications to core fuzzing algorithms.

D. AUGUR's Domain-specific Language (DSL)

We now present AUGUR's property DSL and its efficient verification. **Syntax.** More generally, the DSL consists of a sequence of *sort annotations* followed by a sequence of *formulas*. Each *sort* (i.e., simple type) annotation is of the form $\tau \text{ id}_0 [\{ \text{id}_i^+ \}]$, where τ denotes the sort and each `id` is an identifier (`id0` denotes the variable name, and `idi+` denotes one or more enum constants). The square brackets denote that the enum items (enclosed in curly braces) are optional (they should only be present when the $\tau = \text{enum}$). For simplicity, we can consider $\tau \in \{\text{int}, \text{bool}, \text{enum}\}$, but the DSL can naturally be extended to consider a wider range of sorts. Each formula is a QF-FO-PLTL formula from the following BNF.

$$\begin{aligned} \varphi &::= \top \mid \perp \mid A \mid (t = t) \mid \neg\varphi \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid \\ &\quad (\varphi \Rightarrow \varphi) \mid Y \varphi \mid O \varphi \mid H \varphi \mid (\varphi S \varphi) \\ A &::= p(t, \dots, t) \mid t \bowtie t \mid \circ t \\ t &::= x \mid c \mid f(t, \dots, t) \end{aligned}$$

$\bowtie$ denotes binary operators over terms (e.g., $+$), and $\circ$ denotes unary operators over terms (e.g., unary $-$). We assume that every free variable x is ascribed exactly one sort. We also assume that every predicate symbol p , function symbol f , and constant symbol c has a fixed sort of the form $\tau_1 \times \dots \times \tau_n \rightarrow \tau_{n+1}$, where τ_{n+1} is `bool` for predicate symbols. With the sort annotations, we can check all sorts bottom-up, and hence we only consider well-sorted formulas.

Semantics. We define the DSL semantics as the set of traces that are flagged as violating one or more properties, where each trace is defined as an alternating sequence of input and output network packets sent along the wire. Below, we use σ to denote a *trace*, which is a finite sequence of messages $m_1, \dots, m_n$, where each m_i is a sequence of bits corresponding to a concrete fuzzer-generated or server-generated packet processed by the `MONITOR`. We use $\sigma(i)$ to denote m_i (i.e., the i th packet in the trace). Further, the semantics is parametric with respect to the predicate evaluator, which we denote using $\llbracket \cdot, \cdot \rrbracket$. In other words, $\llbracket \cdot, \cdot \rrbracket$ takes as input a predicate and the current packet, and it returns a boolean value, depending on the protocol-specific implementation of the predicate evaluator. We write $(\sigma, i, \llbracket \cdot, \cdot \rrbracket) \models \varphi$ to denote that φ holds at position i

of trace σ using predicate evaluator $\llbracket \cdot, \cdot \rrbracket$, where $\models$ is defined as follows (P shorthand for $p(t_1, \dots, t_n)$):

$$\begin{aligned} (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models \top \\ (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models P && \text{if } \llbracket P, \sigma(i) \rrbracket \\ (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models \neg\varphi && \text{if } (\sigma, i, \llbracket \cdot, \cdot \rrbracket) \not\models \varphi \\ (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models \varphi_1 \wedge \varphi_2 && \text{if } (\sigma, i, \llbracket \cdot, \cdot \rrbracket) \models \varphi_1 \text{ and } \\ &&& (\sigma, i, \llbracket \cdot, \cdot \rrbracket) \models \varphi_2 \\ (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models Y \varphi && \text{if } i > 1 \text{ and } (\sigma, i-1, \llbracket \cdot, \cdot \rrbracket) \models \varphi \\ (\sigma, i, \llbracket \cdot, \cdot \rrbracket) &\models \varphi_1 S \varphi_2 && \text{if } \exists j \leq i. (\sigma, j, \llbracket \cdot, \cdot \rrbracket) \models \varphi_2 \text{ and } \\ &&& \forall k. j < k \leq i. (\sigma, k, \llbracket \cdot, \cdot \rrbracket) \models \varphi_1 \end{aligned}$$

The remaining connectives and temporal operators can be implemented by desugaring to the ones defined above. Notice that with our predicate evaluator $\llbracket \cdot, \cdot \rrbracket$, the semantics essentially reduces to propositional (past-only) LTL. From this, we can define the semantics of a DSL input as the set of finite traces violating some property (where $\text{Prop}(\mathcal{D})$ denotes the set of properties in $\mathcal{D}$):

$$\llbracket \mathcal{D} \rrbracket_{\text{viol}} = \{ \sigma \mid \exists i \leq |\sigma|. \exists \varphi \in \text{Prop}(\mathcal{D}). (\sigma, i, \llbracket \cdot, \cdot \rrbracket) \not\models \varphi \}.$$

Runtime verification algorithm. As there are no quantifiers in our properties, we can use the dynamic programming algorithm of runtime verification developed for the propositional fragment of past-only LTL (PLTL) [6]. For a property ψ and a given trace $\sigma \cdot s$ where $\cdot$ denotes the concatenation of a trace with a new state s , the algorithm maintains a bit indicating whether each sub-formula of ψ is satisfied by the trace σ . Based on this state, the algorithm can incrementally update the satisfaction status of ψ when a new state s is added to the trace σ . For a given trace σ , we use $\sigma[\cdot i]$ to denote the prefix of σ up to the i -th state. Abusing notation, let $\llbracket \phi \rrbracket^i$ denote the Boolean truth value of ϕ when evaluated on $\sigma[\cdot i]$ (i.e., the trace prefix of σ from s_1 to s_i , inclusive). One can inductively calculate this value in the following way (assume $i \geq 1$):

$$\begin{aligned} \llbracket \top \rrbracket^i &= \text{True} \\ \llbracket \perp \rrbracket^i &= \text{False} \\ \llbracket p(t_1, \dots, t_n) \rrbracket^i &= \llbracket p(t_1, \dots, t_n), \sigma(i) \rrbracket \\ \llbracket \neg\varphi \rrbracket^i &= \neg \llbracket \varphi \rrbracket^i \\ \llbracket \varphi_1 \wedge \varphi_2 \rrbracket^i &= \llbracket \varphi_1 \rrbracket^i \wedge \llbracket \varphi_2 \rrbracket^i \\ \llbracket Y \varphi \rrbracket^i &= \llbracket \varphi \rrbracket^{(i-1)} \text{ if } i > 1 \\ \llbracket Y \varphi \rrbracket^i &= \text{False} \text{ if } i = 1 \\ \llbracket \varphi_1 S \varphi_2 \rrbracket^i &= \llbracket \varphi_2 \rrbracket^i \vee (\llbracket \varphi_1 S \varphi_2 \rrbracket^{(i-1)} \wedge \llbracket \varphi_1 \rrbracket^i) \end{aligned}$$

In the predicate (third) case above, the notation $\llbracket p(t_1, \dots, t_n), \sigma(i) \rrbracket$ on the right-hand side refers to `PRED-EVALUATOR`. Note that this evaluation only ever references single-bit states at timesteps i and $i-1$ and *not* on the entire trace history, enabling monitor to store only a constant amount of state per subformula.

V. IMPLEMENTATION OF AUGUR

We integrate AUGUR with `Saeced` [39], `ResolverFuzz` [52], `AFLNET` on `ProFuzzBench` [38], [35] and `SNPSFuzzer`

[25]. We have implemented the core components of AUGUR in C/C++ with 1566 SLOC. Each fuzzer-protocol pair has its own FIELDEXTRACTOR.

Predicate evaluator. The particular predicates used in the property specification impact the complexity of the PREDEVALUATOR. If one uses coarse-grained predicates the specification, for example, a predicate that checks whether the current message is well-formed or not ($\text{IsPacketWellFormed}(m_i)$), then the predicate can be evaluated with any packet parser. However, if one were to choose fine-grained predicates in their specification, then it enables more fine-grained root cause detection for violations. For example, instead of the predicate $\text{IsPacketWellFormed}(m_i)$, one could consider different fine-grained predicates stipulating whether the individual fields of the packet are (semantically) well-formed (e.g., $\text{IsRejectedGrpLengthCorrect}(m_i)$). However, this comes at a price at implementing a more complicated PREDEVALUATOR.

Scoring Flagged Inputs. A desirable feature of AUGUR is that it can support different user-specific prioritization of the flagged inputs. AUGUR can output the final runtime verification state of all flagged inputs for all the monitored expected protocol properties. This can be conceptually viewed as a binary matrix, where each row corresponds to a flagged input and each column corresponds to an expected property. The value of each cell in the matrix is either **1** or **0**, indicating whether the input violates the property or not. There are many ways to use the information in this matrix to provide a score to each flagged input, and based on the score, one can prioritize the flagged inputs and present it to the user. For example, one user may be interested in prioritizing the flagged inputs based on the number of properties violated by the input, while another user may be interested in prioritizing the flagged inputs based on the severity of the violated properties. Others may prefer to analyze the smallest trace that violates a single, high-severity property.

We use the following criteria to order flagged inputs in AUGUR when incorporated with FUZZERS_{PS} and FUZZERS_{PA}. We provide a weight reflecting the importance of each expected property. Each flagged input can then be assigned to a unique bucket based on the properties it violates. The buckets are then ordered based on the weight of the properties. Inside each bucket, the flagged inputs can be ordered based on their lengths.

Snapshot-based reset support. AUGUR supports fuzzers that use snapshot-based resetting techniques by design. A key challenge with snapshot-based resetting is that the MONITOR state may become inconsistent with the actual SUT state after a reset. To avoid replaying the entire execution trace, AUGUR provides an API that stores the MONITOR state bit vectors at each snapshot point. Upon reset, the fuzzer invokes this API to restore the MONITOR state to the last snapshot point, by maintaining snapshot identifiers. This ensures that the MONITOR state remains consistent with the SUT state after a reset, allowing AUGUR to continue runtime verification accurately.

Minimization scripts. By definition, once a safety property

Fuzzer	Protocol	Target(s)	Mode
SAECRED	WiFi	hostap ASUS TUF ASUS 1800S TP-Link	WPA3 Only
ResolverFuzz	DNS	BIND9 Unbound PowerDNS Knot MaraDNS Technitium	Forward-Only
AFLNET	FTP RTSP DNS SSH DTLS	LightFTP, BFTPD, PureFTPD Live555 dnsmasq OpenSSH tinyDTLS	NA
SNPSFuzzer	FTP DNS DTLS	LightFTP – dnsmasq tinyDTLS	NA

TABLE II: Fuzzers, protocols and implementations evaluated. Mode refers to the implementation’s operational mode. NA indicates that the target does not require a specific operational mode.

is violated on some finite trace σ , it must also be violated for every possible extension of σ (i.e., $\sigma \cdot \sigma'$ for some σ'). This can pollute logs: once a trace σ is violated, the violation will be re-logged for every future extension of σ . This can be addressed with straightforward *minimization scripts* that remove violations from the log whose last timestep does not violate some new property φ .

VI. EVALUATION

We now discuss the evaluation setup as well as the results of property-aware bug detection approach. Table II summarizes the fuzzers, protocols and targets evaluated. For Saecred, we use the operational mode used by the authors in their evaluation [39], and for ResolverFuzz, we use one of the 4 operational modes. For the sake of comparison, we will report our findings in light of the results from the original evaluation modes of these fuzzers. These fuzzers ran in the range of 7 – 24 hours across all implementations, with variations due to fuzzer-unreliability (AFLNET crashed multiple times against certain implementations whilst campaigning, requiring restarts) and fuzzer-specific issues (ResolverFuzz parameters affect fuzzing execution time). In total, we ran AUGUR $\approx$ 344 hours across all fuzzers and implementations.

This choice of fuzzers allows us to generalize the efficacy of AUGUR across fuzzers that (1) have access to source code; (2) are completely black-box; (3) leverage stateful protocol semantics through state-inference engines, reference implementation oracles or differential testing; (4) rely on lightweight instrumentation to guide the execution flow; and (5) have snapshot and restore capabilities to speed up fuzzing runtimes.

To showcase the interoperability, usability and efficacy of AUGUR, we answer the following research questions:

- **RQ1.** How interoperable is AUGUR?
- **RQ2.** What is the required effort to integrate AUGUR against a fuzzer-protocol pair?
- **RQ3.** How effective is AUGUR in identifying logical bugs and assisting in triaging across different fuzzers and implementations?

Answering RQ1. One of the our key design goals is interoperability for a diverse set of fuzzers and protocols. To answer RQ1, we evaluate AUGUR against the four fuzzers and run against 17 implementations. We will show through RQ3 that AUGUR is effective in identifying logical bugs across all fuzzers, thereby answering RQ1.

Answering RQ2. To answer RQ2, we measure the integration effort required to port AUGUR to a fuzzer-protocol pair. To this end, we measure the (1) SLOC required to implement the PREDICATEEVALUATOR for each fuzzer-protocol pair; (2) the SLOC required to hook the fuzzer input-output loop, reset the evaluator environment between test-cases, and reset the bit vectors for snapshot and restore capabilities. We also report the estimated time taken to implement these components for each fuzzer-protocol pair.

Answering RQ3. In Section VIII, we report the semantic bugs identified by AUGUR across the four fuzzers. To measure AUGUR’s efficacy, we report the number of distinct bugs identified across all fuzzers and implementations, along with a breakdown of bugs per fuzzer and implementation. Of the total distinct bugs identified, we discuss the security implications of the 6 previously unknown bugs. This analysis will show AUGUR’s impact in lifting FUZZERS_{PA}. Additionally, we triage the bugs identified by AUGUR against Saecred and will show the reduction in false-positives when compared to Saecreds reported deviations. This analysis will show the efficacy of runtime verification in reducing false-positives when compared to testing frameworks.

Code coverage and crash-based evaluation. Since AUGUR operates purely as an external observer and does not impact or influence the underlying fuzzer’s execution. Also, due to its applicability in the black-box setting, AUGUR intentionally does not include any mechanism for detecting crashes of the test target. In our evaluation, we do not report code coverage or crash-based metrics as AUGUR has no influence on the underlying fuzzer’s execution other than just discovering violations of the checkable assertions.

VII. INTEGRATION EFFORT (RQ2)

To answer **RQ2**, we quantify the integration effort required to deploy AUGUR along two dimensions: (1) source lines of code (SLOC), and (2) estimated engineering time to implement the FIELDExtractor and insert the API calls at the fuzzer integration points described in Section IV-B.

Table III summarizes the SLOC required to implement the FIELDExtractor for each fuzzer-protocol pair. Table IV reports the additional SLOC needed to hook the fuzzer input-output loop, reset the evaluation environment between test

Module	Lang.	C	H	Total
dnsmasq_predicate_evaluator	C	301	25	326
dtls_predicate_evaluator	C	437	25	462
ftp_predicate_evaluator	C	382	20	402
rtsp_predicate_evaluator	C	386	21	407
ssh_predicate_evaluator	C	273	21	294
ResolverFuzz_predicate_evaluator.py	Python	–	–	560
SAECRED_predicate_evaluator.ml	OCaml	–	–	640
GRAND TOTAL				3,091

TABLE III: SLOC by predicate-evaluator module and implementation language. Columns C and H denote SLOC in the C source and header, respectively.

Fuzzer	Language	SLOC
AFLNET	C	313
SNPSFuzzer	C	334
SAECRED	OCaml	13
ResolverFuzz	Python	354

TABLE IV: SLOC required to hook I/O and invoke APIs for each fuzzer (including saving and restoring bit vectors).

cases, and manage snapshot and restore functionality, including bit vector resets. Across all evaluated systems, the integration effort is dominated by the one-time cost of implementing the FIELDExtractor. In contrast, inserting the API calls incurs relatively little overhead, requiring approximately 300 SLOC in total, of which roughly 200 SLOC correspond to the API implementation. Overall, the integration effort is modest, ranging from a few days to approximately two weeks of engineering effort per fuzzer-protocol pair. In practice, the initial integration of AUGUR with AFLNET across all Pro-FuzzBench Docker images required approximately 10 days, while integrating SNPSFuzzer with snapshot and restore support required about 5 days.

In summary, to answer RQ2, the integration effort ranges from a few hours to several weeks, with unfamiliarity with a given fuzzer or protocol being the primary contributor to longer integration times. Practitioners familiar with both the fuzzer and the target protocol should be able to integrate AUGUR in a matter of hours.

VIII. FINDINGS (RQ1, RQ3)

We now present the results of our evaluation, focusing on the semantic bugs uncovered by AUGUR across all fuzzers. In total, AUGUR uncovered (1) 4 semantic bugs in 3 DNS implementations during the evaluation with ResolverFuzz, (2) 4 semantic bugs in 4 WPA3 implementations during the evaluation with Saecred, (3) 4 semantic bugs in the LightFTP server implementation, out of which 2 were uncovered during the evaluation with AFLNET and 2 during the evaluation with SNPSFuzzer, and (4) 2 semantic bugs in the RTSP Live555 server implementation during the evaluation with AFLNET. Out of these 14 semantic bugs, 6 discovered by AUGUR in LightFTP and Live555 were previously unknown and reproduced in the latest versions of the respective implementations.

We now discuss the details and security implications of these findings and answer RQ1 and RQ3.

A. *LightFTP Semantic Vulnerabilities*

AUGUR uncovered 4 distinct *semantic parsing vulnerabilities* in the LightFTP server. All of these bugs stem from improper input sanitization and validation at the protocol grammar level, allowing malformed FTP commands to be interpreted as valid. Critically, these vulnerabilities enable attackers to trigger security-sensitive protocol behaviors while evading detection by network security devices that assume RFC-compliant traffic. We compiled the latest LightFTP version [27] with AddressSanitizer and confirmed that none of these vulnerabilities lead to memory safety violations, indicating that they are purely semantic in nature.

Null-byte truncation in PASV command. RFC 959 § 5.3 [41] specifies that the PASV command must be terminated by a carriage return and line feed (<CRLF>). LightFTP fails to correctly validate this requirement and accepts truncated commands containing embedded null bytes. AFLNET triggered multiple instances of malformed inputs of the form PA\0, to which LightFTP responds with a successful 227 reply. This behavior violates the FTP specification and allows an attacker to truncate the PASV command via null-byte injection. This enables evasion of intrusion detection and prevention systems (IDS/IPS) that rely on string-based or grammar-based parsing of FTP commands. A compliant implementation must reject these inputs with 500 or 501 error codes.

Successful exploitation allows an attacker to induce the server into opening a listening data port in a manner that is invisible to security devices operating under the assumption of syntactically valid FTP traffic. This discrepancy between server-side interpretation and network-level inspection represents a fundamental semantic gap that can be leveraged to bypass access control and monitoring mechanisms.

Malformed PORT command leading to arbitrary port binding. RFC 959 § 5.3.2 [41] defines the PORT command syntax as exactly six comma-separated decimal values: four bytes encoding the IP address and two bytes encoding the port number, followed by <CRLF>. AFLNET identified multiple malformed inputs of the form PORT <IP><PORT_NO>GARBAGE, where extraneous trailing data violates the specification. LightFTP incorrectly parses these malformed commands and proceeds to bind to the specified port number, silently ignoring the trailing garbage. A compliant FTP server must reject such inputs with 50X error codes. Instead, LightFTP transitions into a security-critical state based on invalid inputs.

This vulnerability has serious security implications. By accepting malformed PORT commands, LightFTP enables attackers to bypass network security devices and mount FTP bounce attacks [1]. Because the malformed commands do not conform to the expected grammar, they may evade detection by middleboxes that enforce RFC-compliant parsing, while still being accepted by the server. This highlights a class

of semantic vulnerabilities that cannot be detected through syntactic validation alone.

Client CRLF Response Injection I. RFC 959 § 4.1, 4.2 [41] specify that FTP client commands and server replies are delimited by a <CRLF>, and that each server reply consists of a single, atomic line beginning with a three-digit status code. LightFTP violates this invariant when processing a USER command containing an embedded newline. Specifically, the server terminates its response at the position of the embedded newline and emits the remaining portion of the USER argument as a separate control-channel line. As a result, LightFTP generates a syntactically valid FTP reply line whose contents are partially controlled by the client. Because this injected line is emitted by the server itself, it is indistinguishable from a legitimate server response and is therefore trusted by FTP clients and any network devices monitoring the control channel. This enables response injection and spoofing of FTP reply codes, allowing a malicious client to manipulate the client-side state machine and to induce externally visible effects, such as unauthorized data-channel setup or evasion of network security monitoring.

Client CRLF Response Injection II. Client CRLF Response Injection can also be triggered without explicitly embedding a newline character in the USER command. When a USER command is sent immediately followed by an arbitrary, fuzzed command (*i.e.*, with zero inter-packet delay), LightFTP correctly processes the USER command but fails to enforce strict command-response framing on the control channel. Consequently, the server emits a response in which the legitimate reply to the USER command is immediately followed by a second line derived from subsequent client-controlled input. This second line is transmitted as a standalone server response, despite not corresponding to any valid server-side state transition. Because the injected line originates from the server, it is treated as authoritative by FTP clients and network middleboxes. This allows an attacker to influence server-emitted control traffic without violating the syntactic constraints of individual FTP commands, highlighting a semantic vulnerability that cannot be detected by syntactic validation or traditional request-based fuzzing.

B. *Live555 Semantic Vulnerabilities*

AUGUR uncovered 2 distinct *semantic vulnerabilities* in the Live555 RTSP server. These bugs arise from protocol non-compliance and insufficient input validation, leading to deviant but non-crashing behavior. While initially discovered by AUGUR and AFLNET using the ProFuzzBench setup, we reproduced both vulnerabilities on the latest Live555 release obtained from the official repository [29]. As with LightFTP, none of these vulnerabilities were detected by AddressSanitizer, confirming that they do not result in memory safety violations and are purely semantic in nature.

Port-zero SETUP command. RFC 2326 § 10.4, 12.39 [43] specify the valid port ranges that RTSP servers may use for RTP/RTCP data delivery and mandate the use of error responses (*e.g.*, 453 or 461) when session setup fails. AFLNET

generated multiple test cases that violate these requirements and trigger inconsistent session state within Live555. Specifically, the server responds to a malformed SETUP request with status code 200 OK and a transport header containing `server_port=0-0` and `client_port=0-1`.

This behavior indicates a *zombie session initialization*, in which Live555 creates a session object but fails to bind a valid RTP socket. Port zero is reserved and has undefined behavior at the operating system level, and its use in RTSP transport parameters violates the protocol specification. This can lead to client-side misinterpretation of session state, undefined socket behavior, and inconsistent media delivery, highlighting a semantic vulnerability caused by improper protocol-state validation rather than memory corruption.

CSeq echo reflection. RFC 2326 § 9.2, 12.17 [43] define the CSeq header as a strictly numeric sequence identifier used to correlate RTSP requests and responses. AFLNET triggered multiple test cases in which Live555 reflects arbitrary, non-numeric bytes from the request into the CSeq field of the server response. This behavior directly violates the RTSP specification, which mandates that the CSeq value be a valid integer. Because RTSP clients rely on CSeq values to correctly associate responses with outstanding requests, this vulnerability can desynchronize the client-side protocol state machine and undermine assumptions made by RTSP clients, proxies, and monitoring infrastructure. As with the other bugs we identify, this issue does not cause a crash or memory safety violation, but instead exposes a semantic inconsistency that can lead to incorrect or unsafe protocol behavior.

C. SAE Handshake Semantic Vulnerabilities

AUGUR detected all bugs reported by Saecred, including the DoS and downgrade attacks. As Saecred is specifically designed to find these types of attacks, the result is expected. However, AUGUR goes beyond just detecting these attacks, as it classifies the property violations based on their severity. This classification is based on the impact of these attacks on the security of the SAE handshake protocol. Given this, when weighted scoring is applied, traces that trigger these attacks, along with other violations are prioritized and their traces appear at the top of the list. To understand the effectiveness of both runtime verification and weighted scoring, we find that the downgrade attack sits at the top of the report, but going through Saecred’s results, we find the attack trace at position 6,298. This implies that any manual analysis requires significant effort to find the attack trace, whereas AUGUR will instantly report such clear security-critical violations.

D. DNS Resolver Semantic Vulnerabilities

AUGUR uncovered 4 distinct semantic vulnerabilities across 3 DNS resolver implementations during our evaluation with ResolverFuzz. These vulnerabilities were previously reported by ResolverFuzz [52]; however, AUGUR was able to rediscover them independently across multiple implementations without relying on differential testing. This demonstrates AUGUR’s effectiveness as a black-box technique for rapid

bug triaging and validation across diverse resolver implementations.

AUGUR identified the following 4 semantic vulnerabilities in Unbound: ignoring the RD flag, fragmentation-based cache poisoning, invalid cache entry storage, and acceptance of unsolicited DNS records. In PowerDNS, AUGUR identified 1 vulnerability corresponding to ignoring the RD flag. In Knot Resolver, AUGUR identified 2 vulnerabilities: ignoring the RD flag and fragmentation-based cache poisoning.

Answering RQ1 and RQ3. AUGUR was successfully integrated with all fuzzers, and uncovered numerous security critical semantic bugs. It uncovered all bugs reported by Saecred in the WPA3 implementations and uncovered 4 bugs in multiple DNS resolver implementations, with results similar to those reported by ResolverFuzz. AUGUR also successfully uncovered security critical protocol violations in FTP and RTSP server implementations, in cases where (1) the discovered vulnerabilities were previously unknown, (2) the vulnerabilities were not detected by AddressSanitizer, confirming that they could not be detected by vanilla AFLNET or SNPSFuzzer, and (3) successfully lifted AFLNET and SNPSFuzzer to discover semantic bugs despite their incapability to do so natively.

These findings demonstrate both AUGUR’s interoperability among a diverse set of fuzzers, protocols and implementations (answering RQ1), as well as its effectiveness in identifying logical bugs and assisting in triaging across different fuzzers and implementations (answering RQ3).

IX. RELATED WORK

In the realm of runtime verification, two notable works are relevant to AUGUR, Proteus [44] and LTL-Fuzzer [33]. Proteus is a stateful protocol testing approach that leverages protocol automaton specifications to reduce the wastage of inputs during protocol testing. Proteus, however, is limited in its ability to produce invalid inputs, restricted to fine-grained deviations from the protocol automaton. As a result, it too is ineffective at adequate exploration of the input space. AUGUR differentiates itself from Proteus by relying on runtime verification of temporal properties not for input generation, but rather for existing fuzzer extension. This allows AUGUR to benefit from the input space exploration capabilities of FUZZERS_{PS} and FUZZERS_{PA}, while still being able to identify semantic bugs through runtime verification.

LTL-Fuzzer [33] employs linear temporal logic as a *guidance mechanism* for fuzzing by translating LTL specifications into Büchi automata and steering program execution toward accepting states via instrumenting the SUT. This design enables expressive reasoning over both safety and liveness properties, but incurs substantial space and computational overhead. In particular, automaton-based runtime verification requires maintaining automaton state, handling cycle detection, and tracking execution history, which introduces nontrivial runtime costs and tightly couples the approach to white-box access and program-specific instrumentation. As a result, LTL-Fuzzer is best suited for single instrumentable implementations

and does not generalize to black-box settings or protocol implementations where internal state is unavailable.

In contrast, AUGUR uses LTL exclusively for *runtime verification and triage*, rather than for guiding input generation. By restricting to quantifier-free, past-only, first-order LTL over observable protocol I/O, AUGUR enables incremental runtime verification via a dynamic-programming-based algorithm with $\mathcal{O}(|\varphi|)$ *time per event and constant memory per subformula*. This design avoids automaton construction, cycle detection, and history retention, yielding significantly lower space and compute overhead, making AUGUR compatible with long-running fuzzing campaigns, snapshot-based fuzzers, and high-throughput protocol testing. The trade-off is that AUGUR focuses on safety properties rather than unbounded liveness; however, bounded liveness properties, which are sufficient in fuzzing contexts, can be naturally encoded as safety properties. This deliberate restriction allows AUGUR to remain lightweight, fuzzer-agnostic, and fully black-box, enabling semantic bug detection and prioritization across multiple protocol implementations with a single integration effort per fuzzer-protocol pair.

AUGUR positions itself as a runtime verification approach to be used in conjunction with FUZZERS_{PA} and FUZZERS_{PS} for stateful protocol implementations. To the best of our knowledge, AUGUR is the first approach to assist existing FUZZERS_{PA} in semantic bug detection and FUZZERS_{PS} in triaging their results based on protocol-specific temporal properties, thus increasing the effectiveness of both fuzzing classes at uncovering high-value bugs in stateful protocol implementations.

X. DISCUSSION

Protocol assumptions. AUGUR assumes that the protocol under test adheres to sequential synchronous I/O semantics: it processes one request at a time and responds before processing the next request. While this assumption holds for many protocols, there are certain protocols that employ asynchronous I/O semantics or pipelining, where multiple requests can be processed concurrently. AUGUR’s current design is not directly applicable in those cases.

Fuzzer assumptions. ResolverFuzz’s DNS fuzzing approach operates under a three-party model, where the fuzzer plays a dual-role: one as an attacker-client and another as an attacker-server. This model is not what AUGUR was initially designed for, and required adaptations to monitor the client and implementation server interactions exclusively. While this adaptation was possible due to AUGUR’s modular design and abstract representation of protocol semantics, it requires careful consideration when integrating AUGUR with fuzzers that employ non-standard models.

XI. CONCLUSION

We presented AUGUR, a runtime verification framework for stateful network protocol fuzzers. Fuzzing stateful protocols presents unique challenges, with existing fuzzers leveraging various techniques to minimize test-case generation time

and maximize input space exploration. Through the use of specification oracles, AUGUR gives crash-based fuzzers the ability to identify logical bugs in protocol implementations, while also assisting protocol-specific fuzzers in triaging their results based on protocol-specific temporal properties, reducing their reliance on reference implementation oracles and differential testing, and providing a more scalable and effective approach to bug triaging. This paper is a best-effort attempt at selecting a diverse set of fuzzers and network-protocol implementations to demonstrate AUGUR’s ability. Overall, AUGUR represents a significant step forward in automating semantic bug detection and triaging for stateful protocols, providing a valuable tool for protocol developers and testers to identify and remediate security-critical bugs more efficiently, whilst reduced overhead and engineering efforts required to deploy semantic bug detection capabilities across a wide range of fuzzers and protocols.

REFERENCES

- [1] Mark Allman and Shawn Ostermann. FTP Security Considerations. RFC 2577, May 1999. URL: <https://www.rfc-editor.org/info/rfc2577>, doi:10.17487/RFC2577.
- [2] Bowen Alpern and Fred B Schneider. Recognizing safety and liveness. *Distributed computing*, 2(3):117–126, 1987.
- [3] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. Nautilus: Fishing for deep bugs with grammars. *Proceedings 2019 Network and Distributed System Security Symposium*, 2019. URL: <https://api.semanticscholar.org/CorpusID:69790362>.
- [4] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. Stateful greybox fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA, August. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/ba>.
- [5] Howard Barringer, Allen Goldberg, Klaus Havelund, and Koushik Sen. Program monitoring with ltl in eagle. In *18th International Parallel and Distributed Processing Symposium, 2004. Proceedings.*, page 264. IEEE, 2004.
- [6] David Basin, Matúš Harvan, Felix Klaedtke, and Eugen Zălinescu. Monpoly: monitoring usage-control policies. In *Proceedings of the Second International Conference on Runtime Verification, RV’11*, page 360–364, Berlin, Heidelberg, 2011. Springer-Verlag. doi:10.1007/978-3-642-29860-8_27.
- [7] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. A messy state of the union: Taming the composite state machines of tls. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy, SP ’15*, USA, 2015. IEEE Computer Society. doi:10.1109/SP.2015.39.
- [8] Fabian Bäumer, Marcus Brinkmann, Nurullah Erinola, Sven Hebrok, Nico Heitmann, Felix Lange, Marcel Maehren, Robert Merget, Niklas Niere, Maximilian Radoy, et al. Tls-attacker: A dynamic framework for analyzing tls implementations. *Proceedings of Cybersecurity Artifacts Competition and Impact Award (ACSAC’24)*, 2024.
- [9] Hongjian Cao, Lin Huang, Shuwei Hu, Shangcheng Shi, and Yujia Liu. Owfuzz: Discovering wi-fi flaws in modern devices through over-the-air fuzzing. In *Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec ’23*, page 263–273, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3558482.3590174.
- [10] Sze Yiu Chau, Omar Chowdhury, Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. Symcerts: Practical symbolic execution for exposing noncompliance in x.509 certificate validation implementations. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 503–520, 2017. doi:10.1109/SP.2017.40.

- [11] Yuanliang Chen, Fuchen Ma, Yuanhang Zhou, Yu Jiang, Ting Chen, and Jianguang Sun. Tyr: Finding consensus failure bugs in blockchain system with behaviour divergent model. *2023 IEEE Symposium on Security and Privacy (SP)*, 2023. URL: <https://api.semanticscholar.org/CorpusID:260004028>.
- [12] Giuseppe De Giacomo, Antonio Di Stasio, Francesco Fuggitti, and Sasha Rubin. Pure-past linear temporal and dynamic logic on finite traces. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI'20*, 2021.
- [13] Joyanta Debnath, Christa Jenkins, Yuteng Sun, Sze Yiu Chau, and Omar Chowdhury. Armor: A formally verified implementation of x.509 certificate chain validation. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1462–1480, 2024. doi:10.1109/SP54263.2024.00220.
- [14] Zakir Durumeric, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, and J. Alex Halderman. The matter of heartbleed. In *Proceedings of the 2014 Conference on Internet Measurement Conference, IMC '14*, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2663716.2663755.
- [15] Martin Eberlein, Yannic Noller, Thomas Vogel, and Lars Grunske. Evolutionary grammar-based fuzzing. In Aldeida Aleti and Annibale Panichella, editors, *Search-Based Software Engineering*. Springer International Publishing, 2020.
- [16] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020. URL: <https://www.usenix.org/conference/woot20/presentation/fioraldi>.
- [17] Paul Fiterau-Brosteau, Bengt Jonsson, Robert Merget, Joeri de Ruiter, Konstantinos Sagonas, and Juraj Somorovsky. Analysis of DTLS implementations using protocol state fuzzing. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2523–2540. USENIX Association, August 2020. URL: <https://www.usenix.org/conference/usenixsecurity20/presentation/fiterau-brosteau>.
- [18] Matheus E. Garbelini, Vaibhav Bedi, Sudipta Chattopadhyay, Sumei Sun, and Ernest Kurniawan. BrakTooth: Causing havoc on bluetooth link manager via directed fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/garbelini>.
- [19] Matheus E. Garbelini, Chundong Wang, and Sudipta Chattopadhyay. Greyhound: Directed greybox wi-fi fuzzing. *IEEE Transactions on Dependable and Secure Computing*, 2022. doi:10.1109/TDSC.2020.3014624.
- [20] Hugo Gascon, Christian Wressnegger, Fabian Yamaguchi, Daniel Arp, and Konrad Rieck. Pulsar: Stateful black-box fuzzing of proprietary network protocols. 2015.
- [21] Syed Rafiul Hussain, Imtiaz Karim, Abdullah Al Ishtiaq, Omar Chowdhury, and Elisa Bertino. Noncompliance as deviant behavior: An automated black-box noncompliance checker for 4g lte cellular devices. In *ACM SIGSAC Conference on Computer and Communications Security, CCS '21*, 2021.
- [22] Imtiaz Karim, Fabrizio Cicala, Syed Rafiul Hussain, Omar Chowdhury, and Elisa Bertino. Opening pandora's box through atfuzzer: dynamic analysis of at interface for android smartphones. In *Proceedings of the 35th Annual Computer Security Applications Conference, ACSAC '19*, 2019. doi:10.1145/3359789.3359833.
- [23] Kyungtae Kim, Sungwoo Kim, Kevin R. B. Butler, Antonio Bianchi, Rick Kennell, and Dave (Jing) Tian. Fuzz the power: Dual-role state guided black-box fuzzing for USB power delivery. In *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/kim-kyungtae>.
- [24] Junqiang Li, Senyi Li, Gang Sun, Ting Chen, and Hongfang Yu. Snpsfuzzer: A fast greybox fuzzer for stateful network protocols using snapshots. *IEEE Transactions on Information Forensics and Security*, 2022. doi:10.1109/TIFS.2022.3192991.
- [25] Junqiang Li, Senyi Li, Gang Sun, Ting Chen, and Hongfang Yu. Snpsfuzzer: A fast greybox fuzzer for stateful network protocols using snapshots. *IEEE Transactions on Information Forensics and Security*, 17:2673–2687, 2022. doi:10.1109/TIFS.2022.3192991.
- [26] Orna Lichtenstein, Amir Pnueli, and Lenore D. Zuck. The glory of the past. In *Proceedings of the Conference on Logic of Programs*, page 196–218, Berlin, Heidelberg, 1985. Springer-Verlag.
- [27] LightFTP Project. LightFTP: Small x86-32/x64 FTP Server, 2024. URL: <https://github.com/hfirefox/LightFTP>.
- [28] Dongge Liu, Van-Thuan Pham, G. Ernst, Toby C. Murray, and Benjamin I. P. Rubinstein. State selection algorithms and their impact on the performance of stateful network protocol fuzzing. *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. URL: <https://api.semanticscholar.org/CorpusID:245634753>.
- [29] Inc. Live Networks. Live555 Streaming Media (livemedia) library. <http://live555.com/liveMedia/>. Accessed: 2026-02-06.
- [30] Zhengxiong Luo, Junze Yu, Feilong Zuo, Jianzhong Liu, Yu Jiang, Ting Chen, Abhik Roychoudhury, and Jianguang Sun. Bleem: packet sequence oriented fuzzing for protocol implementations. In *USENIX Security '23*, SEC '23.
- [31] Kenneth L. McMillan and Lenore D. Zuck. Formal specification and testing of QUIC. In *Proceedings of ACM Special Interest Group on Data Communication (SIGCOMM'19)*. ACM, 2019. to appear. URL: <https://doi.org/10.1145/3351000.3351082>.
- [32] Ruijie Meng, Zhen Dong, Jialin Li, Ivan Beschastnikh, and Abhik Roychoudhury. Linear-time temporal logic guided greybox fuzzing. In *Proceedings of the 44th International Conference on Software Engineering, ICSE '22*, page 1343–1355, New York, NY, USA, 2022. Association for Computing Machinery. doi:10.1145/3510003.3510082.
- [33] Ruijie Meng, Zhen Dong, Jialin Li, Ivan Beschastnikh, and Abhik Roychoudhury. Linear-time temporal logic guided greybox fuzzing. In *Proceedings of the 44th International Conference on Software Engineering, ICSE '22*, 2022. URL: <https://doi.org/10.1145/3510003.3510082>.
- [34] Roberto Natella. Stateful: Greybox fuzzing for stateful network servers. *Empirical Software Engineering*, 27, 2022. doi:10.1007/s10664-022-10233-3.
- [35] Roberto Natella and Van-Thuan Pham. Profuzzbench: a benchmark for stateful protocol fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021*, page 662–665, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3460319.3469077.
- [36] CheolJun Park, Sangwook Bae, BeomSeok Oh, Jiho Lee, Eunkyoo Lee, Insu Yun, and Yongdae Kim. DoLTEst: In-depth downlink negative testing framework for LTE devices. In *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA, 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/park-cheoljun>.
- [37] Haram Park, Carlos Nkuba Kayembe, Seunghoon Woo, and Heejo Lee. L2fuzz: Discovering bluetooth l2cap vulnerabilities using stateful fuzz testing. *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2022. URL: <https://api.semanticscholar.org/CorpusID:251087626>.
- [38] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. Afnets: A greybox fuzzer for network protocols. *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 460–465, 2020. URL: <https://api.semanticscholar.org/CorpusID:220881956>.
- [39] Muhammad Daniyal Pirwani Dar, Rob Lorch, Aliakbar Sadeghi, Vincenzo Sorcigli, Héloïse Gollier, Cesare Tinelli, Mathy Vanhoef, and Omar Chowdhury. Saecred: A state-aware, over-the-air protocol testing approach for discovering parsing bugs in sae handshake implementations of cots wi-fi access points. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3691–3709, 2025. doi:10.1109/SP61157.2025.00211.
- [40] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pages 46–57, 1977. doi:10.1109/SFCS.1977.32.
- [41] J. Postel and J. Reynolds. File transfer protocol. Technical Report RFC 959, Internet Engineering Task Force (IETF), October 1985. URL: <https://www.rfc-editor.org/rfc/rfc959.txt>.
- [42] Shisong Qin, Fan Hu, Zheyu Ma, Bodong Zhao, Tingting Yin, and Chao Zhang. Nsfuzz: Towards efficient and state-aware network service fuzzing. *ACM Trans. Softw. Eng. Methodol.*, 32(6), September 2023. doi:10.1145/3580598.
- [43] Anup Rao, Rob Lanphier, and Henning Schulzrinne. Real Time Streaming Protocol (RTSP). RFC 2326, April 1998. URL: <https://www.rfc-editor.org/info/rfc2326>, doi:10.17487/RFC2326.
- [44] Syed Md Mukit Rashid, Tianwei Wu, Kai Tu, Abdullah Al Ishtiaq, Ridwanul Hasan Tanvir, Yilu Dong, Omar Chowdhury, and Syed Rafiul Hussain. State machine mutation-based testing framework for wireless communication protocols. In *Proceedings of the 2024 on ACM SIGSAC*

- Conference on Computer and Communications Security*, CCS '24, page 2102–2116, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690312.
- [45] Kostya Serebryany. OSS-Fuzz - google's continuous fuzzing service for open source software. Vancouver, BC, August 2017. USENIX Association.
 - [46] Mathy Vanhoef and Frank Piessens. Key reinstallation attacks: Forcing nonce reuse in WPA2. In *CCS*, 2017.
 - [47] Moshe Y Vardi and Pierre Wolper. An automata-theoretic approach to automatic program verification. In *Proceedings of the First Symposium on Logic in Computer Science*. Institute of Electrical and Electronics Engineers (IEEE), 1986. arXiv:<https://orbi.uliege.be/handle/2268/116609>.
 - [48] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-aware greybox fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019.
 - [49] Moosa Yahyazadeh, Sze Yiu Chau, Li Li, Man Hong Hue, Joyanta Debnath, Sheung Chiu Ip, Chun Ngai Li, Endadul Hoque, and Omar Chowdhury. Morpheus: Bringing the (pkcs) one to meet the oracle. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2474–2496, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3460120.3485382.
 - [50] Moosa Yahyazadeh, Syed Rafiul Hussain, Endadul Hoque, and Omar Chowdhury. Patriot: Policy assisted resilient programmable iot system. In Jyotirmoy Deshmukh and Dejan Ničković, editors, *Runtime Verification*, pages 151–171, Cham, 2020. Springer International Publishing.
 - [51] Michal Zalewski. Afl (american fuzzy lop). URL: <https://afl-l.readthedocs.io/en/latest/#>.
 - [52] Qifan Zhang, Xuesong Bai, Xiang Li, Haixin Duan, Qi Li, and Zhou Li. ResolverFuzz: Automated discovery of DNS resolver vulnerabilities with Query-Response fuzzing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4729–4746, Philadelphia, PA, August 2024. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-qifan>.
 - [53] Yong-Hao Zou, Jia-Ju Bai, Jielong Zhou, Jianfeng Tan, Chenggang Qin, and Shi-Min Hu. TCP-Fuzz: Detecting memory and semantic bugs in TCP stacks with fuzzing. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, July 2021. URL: <https://www.usenix.org/conference/atc21/presentation/zou>.